

## QUESTION

**It is mentioned in the ID1104 CD. Lesson plan will include knowledge segment(K)of the UNIT 1. It must be completed within 5 hours.**

As this response is AI generated, , it may contain some inaccuracies or the information may sometime be obsolete or incomplete. You are advised to carefully verify the specific details from authentic sources.

At BSDU, we will plan **only the Knowledge segment (K) of Unit I in 5 hours**, using a **mixed method**: concise deductive explanation, discovery through code observation, Think–Pair–Share, guided code tracing, and formative reflection. The application activities **A1–A3** will be addressed separately within the practical component.

**Unit I: Introduction to Python and Basic Syntax**

**Total duration:** 5 hours

**Knowledge segments covered:**

- **K1:** Python environment and `print()` function
- **K2:** Variables, keywords, identifiers, dynamic typing, and dynamic binding
- **K3:** Basic data types and literals
- **K4:** Type conversion between different data formats

**Related CO:** CO1 — Develop Python programs for structured computational problems using variables, data types, expressions, type conversion, and input-output operations.

**Lesson 1: Python Environment and Output Using `print()`**

**Duration:** 60 minutes

**Knowledge focus:** K1 — Basic elements of the Python environment and the use of `print()` for output.

**Lesson Outcomes**

By the end of the lesson, students will be able to:

1. **Identify** the main elements of a Python programming environment.
2. **Interpret** the purpose and output of simple Python `print()` statements.
3. **Differentiate** between Python code, comments, and displayed output.

| Time      | Teaching–Learning Flow                                                                                                                                                                         | Faculty Role                                                                               | Student Activity                                                    | Evidence / Formative Check                                                  |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------|
| 0–5 min   | <b>Opening seed:</b> Display a familiar instruction such as “Calculate and display the total bill.” Ask: “How does a computer show its result to a user?”                                      | Pose the context and elicit prior exposure to programming interfaces.                      | Individually note possible ways a program may communicate results.  | Oral responses identifying screen output.                                   |
| 5–12 min  | Introduce Python as a programming language and demonstrate the Python environment: editor, interpreter/terminal, program file, run command, and output area.                                   | Demonstrate the environment using VS Code, Python IDLE, or another approved Python editor. | Observe, label the visible environment components in a worksheet.   | Students identify editor, code area, terminal/output area, and run command. |
| 12–20 min | Demonstrate basic <code>print()</code> statements with text, numbers, and arithmetic expressions. Ask students to predict output before execution.                                             | Explain syntax briefly; use prediction before revealing output.                            | Think–Pair–Share: predict output of three statements.               | Responses to output-prediction prompts.                                     |
| 20–30 min | <b>Discovery activity:</b> Show pairs of correct and incorrect <code>print()</code> statements. Students identify patterns relating to quotation marks, parentheses, and expression placement. | Facilitate discovery; consolidate the observed syntax rules.                               | Compare code snippets and state why one succeeds or fails.          | Pair worksheet with corrected statements.                                   |
| 30–42 min | Explain output readability: labels, line breaks, and combining text with values at an introductory level.                                                                                      | Demonstrate clear versus unclear output presentation.                                      | Analyse two output samples and identify the more user-friendly one. | One written justification using output terminology.                         |
| 42–52 min | Guided code tracing: students trace a short program containing multiple <code>print()</code> statements and arithmetic expressions.                                                            | Ask probing questions; correct misconceptions.                                             | Write the expected output before running the program.               | Trace-sheet accuracy.                                                       |
| 52–60 min | <b>Summary and reflection:</b> Revisit Python environment, program execution, and <code>print()</code> . End with the question: “What might make a program’s output useful to a real user?”    | Consolidate core points and collect exit tickets.                                          | Submit one <code>print()</code> statement and one doubt/question.   | Exit ticket.                                                                |

## Lesson 2: Variables, Keywords, and Identifiers

**Duration:** 60 minutes

**Knowledge focus:** K2 — Variables, keywords, and identifiers.

### Lesson Outcomes

By the end of the lesson, students will be able to:

1. **Distinguish** variables, identifiers, and Python keywords.
2. **Evaluate** whether a proposed identifier follows Python naming rules.
3. **Interpret** variable assignment statements in simple Python programs.

| Time      | Teaching–Learning Flow                                                                                                                                                                                                            | Faculty Role                                                           | Student Activity                                                           | Evidence / Formative Check        |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------|----------------------------------------------------------------------------|-----------------------------------|
| 0–5 min   | <b>Recap:</b> Review <code>print()</code> through two output-prediction questions.                                                                                                                                                | Prompt recall and connect output with stored values.                   | Respond individually, then compare with a peer.                            | Quick oral check.                 |
| 5–12 min  | <b>Cognitive-dissonance prompt:</b> Display <code>total = 500</code> , <code>class = "BCA"</code> , <code>student-name = "Aman"</code> , and <code>2marks = 90</code> . Ask which statements can run and why.                     | Avoid immediate explanation; invite justification.                     | Classify statements as valid or invalid.                                   | Initial classification responses. |
| 12–25 min | Explain variables as named references to values; introduce assignment operator, identifiers, naming conventions, and keywords.                                                                                                    | Give concise explanation and relate each rule to the opening examples. | Annotate examples: variable name, assignment operator, and assigned value. | Annotated code sheet.             |
| 25–35 min | Think–Pair–Share: provide a list of names such as <code>student_name</code> , <code>totalMarks</code> , <code>for</code> , <code>student-name</code> , and <code>_count</code> . Students classify and correct them where needed. | Facilitate peer discussion and clarify keyword restrictions.           | Classify identifiers and propose improved names.                           | Pair response sheet.              |
| 35–45 min | Demonstrate variable reassignment using a simple example such as <code>score = 75</code> followed by <code>score = 82</code> .                                                                                                    | Connect changed values to later program output.                        | Predict the output after each assignment.                                  | Individual output prediction.     |
| 45–53 min | Guided code-reading task: analyse a short student-profile script and identify variables, identifiers, literals, and keywords.                                                                                                     | Scaffold with one solved example.                                      | Mark elements in the supplied code.                                        | Checked annotation sheet.         |

| Time      | Teaching–Learning Flow                                                                                                                                                                   | Faculty Role                    | Student Activity                                                    | Evidence / Formative Check |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------|---------------------------------------------------------------------|----------------------------|
| 53–60 min | <b>Summary and reflection:</b><br>Consolidate identifier rules and variable assignment. Leave the question: “If a variable can receive a new value, what might happen to its data type?” | Link forward to dynamic typing. | Write one valid identifier, one invalid identifier, and the reason. | Exit ticket.               |

### Lesson 3: Dynamic Typing and Dynamic Binding

**Duration:** 60 minutes

**Knowledge focus:** K2 — Dynamic typing and dynamic binding.

#### Lesson Outcomes

By the end of the lesson, students will be able to:

1. **Trace** changes in a variable’s value and associated data type during program execution.
2. **Analyse** examples that demonstrate dynamic typing and dynamic binding.
3. **Differentiate** between a variable name and the value currently associated with it.

| Time      | Teaching–Learning Flow                                                                                                                  | Faculty Role                                                     | Student Activity                           | Evidence / Formative Check |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------|----------------------------|
| 0–5 min   | <b>Recap:</b> Review variables, identifiers, and reassignment through a short code trace.                                               | Ask students to state the final value of a reassigned variable.  | Predict final output.                      | Oral response.             |
| 5–12 min  | <b>Seed example:</b> Display: <code>`item = 10`, `item = "Book"`, `item = 12.5`</code> . Ask: “Has the name changed? What has changed?” | Encourage observation before introducing terminology.            | Discuss observations in pairs.             | Pair explanation.          |
| 12–24 min | Explain dynamic typing and dynamic binding through visual tracing of name-to-value association.                                         | Use a table or board diagram showing statement, value, and type. | Complete a partially filled tracing table. | Completed trace table.     |
| 24–34 min | Demonstrate <code>`type()`</code> for observing the type of integer, float, string, and Boolean values.                                 | Run sample statements and invite output predictions.             | Predict data type before execution.        | Accuracy of prediction.    |

| Time      | Teaching–Learning Flow                                                                                                                                             | Faculty Role                                                            | Student Activity                                                         | Evidence / Formative Check    |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------|--------------------------------------------------------------------------|-------------------------------|
| 34–45 min | Collaborative analysis: groups receive short code fragments that reassign variables using different values. They identify the value and type after each statement. | Guide groups through questions rather than direct correction.           | Trace program state and present findings.                                | Group tracing sheet.          |
| 45–53 min | Compare fixed-type declarations conceptually with Python’s dynamic handling, without extending into another language’s syntax.                                     | Clarify that the focus is Python behaviour.                             | Write a two-column comparison: “variable name” and “current value/type.” | Individual notebook response. |
| 53–60 min | <b>Summary and reflection:</b> Consolidate dynamic typing and binding. Leave the question: “What kinds of values can Python store, and how can we recognise them?” | Address common confusion between value change and variable name change. | Submit a one-minute explanation of dynamic typing.                       | One-minute paper.             |

## Lesson 4: Basic Data Types and Literals

**Duration:** 60 minutes

**Knowledge focus:** K3 — Basic Python data types and literals.

### Lesson Outcomes

By the end of the lesson, students will be able to:

1. **Classify** Python values as integer, float, string, Boolean, or `None`.
2. **Analyse** the role of literals in Python assignment statements.
3. **Select** appropriate basic data types for simple information items.

| Time     | Teaching–Learning Flow                                                                                                                                           | Faculty Role                          | Student Activity                              | Evidence / Formative Check   |
|----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------|-----------------------------------------------|------------------------------|
| 0–5 min  | <b>Recap:</b> Revisit <code>type()</code> and ask students to identify the type of values from the previous lesson.                                              | Use rapid questioning.                | Respond using hand signals or response cards. | Immediate response check.    |
| 5–12 min | <b>Discovery prompt:</b> Show values such as <code>'25'</code> , <code>25.0</code> , <code>"25"</code> , <code>True</code> , and <code>None</code> . Ask: “Which | Encourage observation and comparison. | Discuss distinctions in pairs.                | Pair-generated observations. |

| Time      | Teaching–Learning Flow                                                                                                                                                   | Faculty Role                                                                    | Student Activity                                                   | Evidence / Formative Check                  |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------|---------------------------------------------|
|           | values look similar but behave differently?"                                                                                                                             |                                                                                 |                                                                    |                                             |
| 12–25 min | Explain integer, float, string, Boolean, and <code>'None'</code> types; introduce literals as directly written values in code.                                           | Present concise definitions with Python examples.                               | Record examples and add one example of each type.                  | Notebook check.                             |
| 25–35 min | Classification task: students sort printed/digital value cards into data-type groups.                                                                                    | Monitor and question ambiguous classifications.                                 | Categorise values collaboratively.                                 | Correctly sorted cards or digital response. |
| 35–44 min | Guided analysis of assignment statements, such as <code>'age = 18'</code> , <code>'name = "Riya"'</code> , and <code>'is_active = True'</code> .                         | Model analysis of one statement.                                                | Identify identifier, literal, and data type in remaining examples. | Worksheet responses.                        |
| 44–53 min | Contextual decision task: select appropriate data types for student name, semester, percentage, attendance status, and optional middle name.                             | Facilitate reasoning and highlight that type selection depends on intended use. | Justify their selections in small groups.                          | Short group explanation.                    |
| 53–60 min | <b>Summary and reflection:</b><br>Consolidate data types and literals. Leave the question: "What should Python do when a value needs to be used in another data format?" | Bridge to type conversion.                                                      | Exit ticket: classify five literals.                               | Exit-ticket accuracy.                       |

## Lesson 5: Type Conversion Between Data Formats

**Duration:** 60 minutes

**Knowledge focus:** K4 — Type conversion between different data formats.

### Lesson Outcomes

By the end of the lesson, students will be able to:

1. **Interpret** the purpose of type conversion in Python.
2. **Trace** the results of conversion using `'int()'`, `'float()'`, and `'str()'`.
3. **Analyse** simple conversion errors caused by incompatible values.

| Time      | Teaching–Learning Flow                                                                                                                                                                                   | Faculty Role                                                                       | Student Activity                                       | Evidence / Formative Check    |
|-----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|--------------------------------------------------------|-------------------------------|
| 0–5 min   | <b>Recap:</b> Ask students to classify <code>"45"</code> and <code>45</code> , then compare their possible use in a calculation.                                                                         | Pose the contrast without resolving it immediately.                                | Predict whether both values behave identically.        | Oral justification.           |
| 5–15 min  | <b>Problem seed:</b> Display <code>marks = "78"</code> and ask why adding <code>marks + 2</code> may not produce the intended numerical result.                                                          | Facilitate discussion and collect hypotheses.                                      | Think–Pair–Share on the cause and possible correction. | Pair response.                |
| 15–28 min | Explain explicit type conversion using <code>int()</code> , <code>float()</code> , and <code>str()</code> . Demonstrate valid conversions and output tracing.                                            | Use short demonstrations and ask prediction questions before execution.            | Trace value and type before and after conversion.      | Conversion-trace table.       |
| 28–38 min | Analyse selected examples including <code>int("25")</code> , <code>float("15.5")</code> , <code>str(90)</code> , and an invalid conversion such as <code>int("twenty")</code> .                          | Explain that conversion depends on whether the source value has a compatible form. | Identify expected result or error for each example.    | Individual response sheet.    |
| 38–48 min | <b>Integrative code reading:</b> Examine a short program that receives text input, converts selected values, and displays labelled output. Focus on understanding, not independent program construction. | Guide students to identify source type, target type, and reason for conversion.    | Annotate conversion statements in the program.         | Annotated code submission.    |
| 48–55 min | Unit-level conceptual review: connect environment, output, variables, dynamic typing, data types, literals, and type conversion through a concept map.                                                   | Facilitate collaborative consolidation.                                            | Build a group concept map and explain one connection.  | Group concept map.            |
| 55–60 min | <b>Summary, formative check, and open-ended question:</b> Short quiz of five questions. Leave the question: "How can these concepts be combined to create a useful interactive Python program?"          | Review quiz responses and preview the practical application segment.               | Complete quiz and write one remaining doubt.           | Quiz and muddiest-point note. |

## Unit I Knowledge-Segment Evidence

By the end of these five hours, students should produce:

- Python-environment component labelling sheet

- `print()` output-prediction sheet
- Identifier and keyword classification worksheet
- Dynamic typing and dynamic binding trace table
- Data-type and literal classification sheet
- Type-conversion trace table
- Unit concept map
- Five-question formative quiz and reflection notes

This five-hour sequence creates the conceptual foundation before students move into input handling, comments, simple Python programs, and analysis of dynamic typing through the Unit I application activities. As Confucius observed, *"I hear and I forget. I see and I remember. I do and I understand."* It reflects BSDU's **Active and Experiential Learning** approach: **Learn, Practice, Perform. Succeed.**

---

Generated by AskSila (asksila.com)